

Technical Summary:

Least-to-Most Prompting Enables Complex Reasoning in Large Language Models

Kai-Yu Lu

1 Research Problem and Motivation

The paper studies how to enable large language models (LLMs) to solve problems that are *harder* than those shown in the prompt examples, with a particular focus on complex reasoning and compositional generalization. The central problem is the *easy-to-hard* generalization gap: LLMs prompted with standard few-shot or chain-of-thought (CoT) examples often fail when test instances require longer reasoning chains or more complex compositions than any training or prompt exemplar.

The motivation is rooted in three gaps between current machine learning systems and human reasoning. First, humans can learn new tasks from very few demonstrations, whereas conventional models typically require large supervised datasets. Second, humans can explain their reasoning process in natural language, while most neural models act as black boxes. Third, humans routinely solve problems that are more difficult than any previously encountered examples, whereas typical machine learning training and test distributions are matched in difficulty. Chain-of-thought prompting narrows the gap by eliciting natural language rationales and improving accuracy on many reasoning benchmarks, but it still struggles when test inputs are substantially more complex than the few-shot exemplars.

The paper therefore investigates whether an explicit form of *problem decomposition* can improve generalization from easy examples to harder test cases by teaching LLMs to build solutions step by step.

2 Related Work

2.1 Few-shot prompting and chain-of-thought reasoning

Chain-of-thought prompting combines few-shot prompting with natural language rationales to improve performance on multi-step reasoning tasks. Prior work demonstrated that, when combined with self-consistency decoding, few-shot chain-of-thought prompts can surpass complicated task-specific architectures trained on much larger labeled datasets, while providing interpretable intermediate rationales. However, chain-of-thought prompts still largely mirror the difficulty of the exemplars and do not explicitly teach a strategy for decomposing problems into easier parts.

2.2 Compositional generalization and SCAN

The SCAN benchmark evaluates compositional generalization by requiring models to map natural language commands to action sequences. The most challenging split is the *length split*, where training action sequences are shorter than those at test time. Existing approaches that achieve near-perfect accuracy on SCAN often rely on neural-symbolic architectures and explicit grammar induction. For example, neural-symbolic stack machines use a neural controller to generate symbolic execution traces, and grammar-based

approaches induce explicit rule sets over a large grammar space. Data augmentation strategies and specialized neural architectures also exist, but length generalization on SCAN remains difficult without symbolic components. The paper positions least-to-most prompting as an alternative that uses general-purpose LLMs without additional training or symbolic modules, yet attains near-perfect accuracy on SCAN, including the length split.

2.3 Easy-to-hard generalization

Beyond SCAN, prior work on easy-to-hard generalization includes neural logic machines and recurrent networks that learn algorithms which scale from small to larger problem instances by performing more recurrent steps at inference time. These models can generalize from small mazes or board configurations to larger ones once a correct algorithm has been learned. The paper extends this line of thought to prompting: instead of modifying the model architecture, the approach modifies how problems are presented to the LLM by teaching it to decompose tasks into easier subproblems.

2.4 Task decomposition and multi-hop question answering

Task decomposition has been explored in multi-hop question answering, where complex questions are split into simpler subquestions that are answered by a base QA model, and the individual answers are aggregated. Other work learns iterative prompting schemes that progressively elicit relevant knowledge from LLMs. There are also systems that chain LLM calls such that outputs of one step are passed as inputs to the next, enabling interactive compositions of reasoning steps. The present paper differs by proposing a two-stage decomposition-and-solving protocol that is itself implemented via few-shot prompting and that emphasizes *dependent* subproblems whose solutions are reused as building blocks.

3 Dataset Construction

3.1 Overview of tasks

The empirical study covers three families of tasks, each probing a different aspect of complex reasoning:

- Symbolic manipulation: the last-letter-concatenation task.
- Compositional generalization: the SCAN benchmark.
- Math word problems: GSM8K and the numerical subset of DROP.

Table 1 summarizes the main datasets and task variants used in the experiments.

3.2 Last-letter-concatenation

The last-letter-concatenation task takes an input list of words and produces the concatenation of the last character of each word. For example, the input list [``thinking'', ``machine''] should produce the output string ``ge''. To evaluate length generalization, the paper constructs test lists with lengths between 4 and 12 by sampling words from a frequency list in Wiktionary. For each list length, 500 lists are created, and exact-match accuracy is reported over these synthetic lists.

3.3 SCAN

SCAN consists of synthetic natural language navigation commands paired with action sequences. Example commands include ``look thrice after jump`` or ``run left and walk``, and the corresponding action sequences are symbolic tokens such as JUMP or TURN LEFT. The experiments focus on the length split, where approximately 80 percent of the full dataset is used for training and contains only shorter action sequences, while test instances contain longer sequences. The paper also examines other splits and the full SCAN dataset, but the core analysis emphasizes the length split as the hardest generalization scenario.

3.4 DROP numeric subset

DROP is a reading comprehension benchmark requiring discrete reasoning over paragraphs. The paper uses the non-football and football subsets and further restricts attention to questions whose answers are numerical. This restriction allows direct reporting of accuracy as the fraction of questions whose predicted numeric answer exactly matches the gold answer. The original DROP dataset is much larger and uses F1 as a standard metric, but the paper focuses on the numeric sub-benchmark where simple exact-match accuracy is appropriate.

3.5 GSM8K

GSM8K is a collection of grade-school math word problems that require multi-step reasoning to compute a numeric answer. The paper does not modify the dataset itself, but it categorizes problems by the number of reasoning steps in the reference solution. This categorization is used later to analyze performance as a function of solution complexity.

4 Query Protocol and Task Definitions

4.1 Prompting paradigms

The study compares several prompting paradigms applied to large language models:

- **Zero-shot prompting:** the model receives only the test input and is asked to answer directly, without any example.
- **Standard few-shot prompting:** prompts contain input-output pairs without intermediate reasoning steps.
- **Chain-of-thought prompting:** prompts contain input, step-by-step natural language reasoning, and final answers for a few demonstration examples.
- **Least-to-most prompting:** the proposed method, which explicitly decomposes a complex problem into simpler subproblems and then solves the subproblems sequentially.

All paradigms interact with LLMs in a black-box manner via text prompts and text completions, without any finetuning.

4.2 Task definitions

For each dataset, the task is defined as follows:

- **Last-letter-concatenation:** input is a list of words; output is the concatenation of the last letter of each word. The model must output exactly the correct string to count as correct.
- **SCAN:** input is a natural language command; output is an action sequence represented either as a sequence of action tokens or as a compact Python-style expression that is later expanded. Exact match between the generated and reference action sequences defines correctness.
- **DROP numeric subset:** input is a paragraph and a question; output is a numeric answer. Accuracy is defined as the fraction of questions whose numeric answer matches the labeled answer.
- **GSM8K:** input is a math word problem; output is a numeric final answer. Again, accuracy is defined as the fraction of problems for which the model outputs the correct number.

4.3 Python-style intermediate representations for SCAN

To respect token limits and keep prompts concise, the SCAN experiments use Python-style expressions as intermediate representations of action sequences. For example, instead of repeating `LOOK LOOK`, the representation `LOOK * 2` is used. These compact expressions are later expanded by a post-processing script to obtain the final action sequences. The paper also shows that LLMs can themselves be prompted to expand such Python expressions with near-perfect accuracy, but this is treated as a supporting experiment rather than the main focus.

5 Modeling Approach

5.1 Base language models

The experiments primarily use the GPT-3 family of models, especially `code-davinci-002`, along with `text-davinci-002` and `code-davinci-001`. The models are treated as fixed black-box LLMs. There is no training or finetuning of parameters; all improvements are achieved purely by carefully designed prompts.

5.2 Least-to-most prompting: high-level idea

Least-to-most prompting is inspired by pedagogical strategies in educational psychology, where a teacher provides a sequence of prompts that gradually move from easier to harder skills. In the context of LLMs, the idea is to instruct the model to:

1. First decompose a complex problem into a sequence of simpler subproblems.
2. Then solve these subproblems in order, using previous intermediate answers as building blocks for subsequent subproblems.

Unlike chain-of-thought prompting, which directly produces a single reasoning chain for the full problem, least-to-most prompting explicitly structures the reasoning as a sequence of dependent queries and responses.

5.3 Two-stage protocol

The canonical least-to-most protocol is defined as a two-stage process.

Stage 1: decomposition. A decomposition prompt contains demonstration exemplars that show how to break a complex input into simpler subproblems. For a new input, the model is asked to produce a list of subproblems. For example, in the last-letter-concatenation task, a long list such as [``thinking`` , ``machine`` , ``learning``] is decomposed into a sequence of shorter sublists that progressively extend the original list. In SCAN, a long command is decomposed into a list of shorter commands.

Stage 2: subproblem solving. A second prompt shows exemplars of how to solve a single subproblem, often in a chain-of-thought style, and how to reuse prior subproblem solutions. For each subproblem in the decomposition, a new query is constructed that contains:

1. The original few-shot exemplars illustrating how to solve subproblems.
2. The list of previously solved subproblems and their answers (if any).
3. The current subproblem that should be solved next.

The model is queried sequentially, and the answer for subproblem i is appended to the context used to solve subproblem $i + 1$. For the last-letter-concatenation task, this setup results in exemplars that illustrate a base case (short list) and a recursive step (longer list that extends the shorter one), and the model learns to reuse previous outputs when constructing the new answer.

5.4 Single-pass variant for math word problems

For GSM8K, the paper uses a variant where decomposition and subproblem solving are combined into a single pass. The prompt first demonstrates how to break a simple word problem into a small number of sub-questions and then immediately solves them in sequence. For a new problem, the model is prompted to both decompose and solve in one completion. The corresponding chain-of-thought baseline is derived by removing the explicit decomposition part while keeping the reasoning trace.

5.5 Baselines and hyperparameters

The baselines include zero-shot prompting, standard few-shot prompting, and chain-of-thought prompting with the same or closely related exemplars. Decoding is performed via the standard LLM interfaces, and the paper focuses on deterministic accuracy numbers rather than exploring a wide variety of sampling hyperparameters. There is no gradient-based optimization, so learning rate, batch size, and training epochs are not applicable. The primary design degrees of freedom consist of:

- The choice and number of exemplars in prompts.
- Whether decomposition and solving stages are separated or merged.
- Whether intermediate answers from previous subproblems are reused in subsequent prompts.

6 Empirical Results

6.1 Symbolic manipulation: last-letter-concatenation

For symbolic manipulation, the paper evaluates standard prompting, chain-of-thought prompting, and least-to-most prompting on the last-letter-concatenation task with test lists of length 4 to 12 using `code-davinci-002`. Standard prompting fails completely, achieving zero accuracy at all lengths. Chain-of-thought prompting yields strong performance for short lists but degrades sharply as list length grows. Least-to-most prompting significantly improves robustness to length:

- At length 4, chain-of-thought prompting obtains approximately 84.2 percent accuracy, whereas least-to-most prompting reaches about 94.0 percent.
- At length 12, chain-of-thought prompting achieves roughly 31.8 percent accuracy, while least-to-most prompting achieves about 74.0 percent.

Thus, least-to-most prompting nearly doubles accuracy at the longest length, and its performance degrades much more slowly as sequence length increases.

Additional experiments in the appendix vary the number of exemplars and the structure of chain-of-thought prompts. Even when chain-of-thought prompting uses more independent examples (for instance, 4 or 8 independent exemplars), it remains far below least-to-most prompting at long lengths. For lists of length 12, enhanced chain-of-thought prompts reach accuracies in the high 30 percent range, still significantly below the 74.0 percent obtained by least-to-most prompting. This supports the claim that the decomposition-and-reuse structure, rather than the sheer number of examples, is responsible for the performance gains.

Error analysis shows that most remaining errors under least-to-most prompting are not due to misidentifying last letters but due to concatenation errors such as dropping a letter or duplicating a letter when combining partial strings. This indicates that the model largely masters the basic sub-tasks (finding last letters) but sometimes fails in the final composition step.

6.2 Compositional generalization: SCAN

For SCAN, the study compares standard prompting, chain-of-thought prompting, and least-to-most prompting on the length split using various GPT-3 models. The key results are:

- With `code-davinci-002`, standard prompting achieves about 16.7 percent accuracy, chain-of-thought prompting achieves about 16.2 percent, while least-to-most prompting reaches approximately 99.7 percent accuracy.
- With `text-davinci-002`, standard prompting obtains roughly 6.0 percent, chain-of-thought prompting 0.0 percent, and least-to-most prompting 76.0 percent.
- With `code-davinci-001`, standard prompting reaches about 0.4 percent, chain-of-thought prompting 0.0 percent, and least-to-most prompting 60.7 percent.

These numbers demonstrate that:

1. Across all base models, least-to-most prompting dramatically outperforms both standard and chain-of-thought prompting.
2. Improvements are especially striking in the most capable base model, where least-to-most prompting essentially solves the length split with near-perfect accuracy.

The experiments also show that when least-to-most prompting is applied to other SCAN splits and the full dataset, the solving rate remains comparable, indicating robust compositional generalization beyond the specific length split. Error analysis reveals that residual failures under least-to-most prompting often stem from subtle semantic misinterpretations, such as incorrectly handling modifiers like “twice” and “thrice” after “around” or confusing the semantics of “after” with “and” when combining sub-sequences.

6.3 Math reasoning: DROP and GSM8K

The paper evaluates math reasoning on the numeric subset of DROP and on GSM8K. The main comparison is between zero-shot, standard prompting, chain-of-thought prompting, and least-to-most prompting using `code-davinci-002`.

For DROP (numeric subsets):

- On the non-football subset, accuracies are approximately 43.86 percent for zero-shot, 58.78 percent for standard prompting, 74.77 percent for chain-of-thought prompting, and 82.45 percent for least-to-most prompting.
- On the football subset, accuracies are about 51.77 percent for zero-shot, 62.73 percent for standard prompting, 59.56 percent for chain-of-thought prompting, and 73.42 percent for least-to-most prompting.

Least-to-most prompting thus provides a substantial improvement over chain-of-thought prompting on DROP, particularly because DROP problems often admit relatively straightforward decompositions into simple sub-questions.

For GSM8K:

- Zero-shot accuracy is about 16.38 percent.
- Standard prompting achieves approximately 17.06 percent.
- Chain-of-thought prompting reaches about 60.87 percent.
- Least-to-most prompting achieves 62.39 percent.

At the aggregate level, least-to-most prompting provides a modest improvement over chain-of-thought prompting on GSM8K, but the step-wise analysis is more revealing. When accuracy is broken down by the number of reasoning steps required by the reference solution:

- For problems requiring two steps, chain-of-thought prompting slightly outperforms least-to-most prompting (around 76.68 percent vs. 74.53 percent).
- For three or four steps, the two methods perform similarly, with small advantages in different regions.
- For problems requiring at least five steps, least-to-most prompting achieves about 45.23 percent, while chain-of-thought prompting reaches only about 39.07 percent.

Therefore, the strength of least-to-most prompting on GSM8K primarily appears in the most complex, multi-step problems, where explicit decomposition brings tangible benefits. The paper further notes that many GSM8K problems that least-to-most prompting fails to solve can be solved when humans provide a correct decomposition manually, highlighting decomposition as the primary bottleneck.

7 Summary

7.1 Methodological contributions

The paper introduces least-to-most prompting as a general prompting strategy for LLMs that consists of two stages: problem decomposition and sequential subproblem solving. Both stages are implemented via few-shot prompting, without any finetuning, architecture changes, or external symbolic modules. The approach draws inspiration from educational psychology and formalizes a “teach the model to decompose, then solve” protocol.

Methodologically, the study demonstrates that:

- Constructing prompts that explicitly reuse intermediate subproblem answers can greatly enhance easy-to-hard generalization in symbolic tasks.
- Combining decomposition prompts with mapping prompts enables LLMs to match or exceed the performance of specialized neural-symbolic systems on SCAN, including the hard length split, while using only a handful of demonstration examples.
- Explicit decomposition can yield consistent gains over chain-of-thought prompting on math reasoning benchmarks, especially on the most complex problems that require many reasoning steps.

7.2 Strengths relative to prior work

Compared with prior work on chain-of-thought prompting and compositional generalization, least-to-most prompting:

- Achieves near-perfect accuracy on SCAN length split with only 14 demonstration examples, while previous neural-symbolic models require full training sets with over fifteen thousand examples.
- Provides a systematic, interpretable decomposition protocol that is implemented entirely at the level of prompts, without domain-specific training or architecture design.
- Demonstrates that careful prompt engineering alone can enable general-purpose LLMs to solve tasks that previously required specialized symbolic machinery.

7.3 Limitations and future directions

The paper explicitly acknowledges several limitations:

- Decomposition prompts are not easily transferable across domains. A prompt that successfully decomposes math word problems does not automatically generalize to common-sense reasoning or other tasks; new prompts must be designed case by case.
- Even within a single domain, constructing decomposition prompts that generalize broadly can be difficult. The experiments suggest that correct decompositions are often sufficient to solve GSM8K problems, but automatically obtaining such decompositions remains challenging.
- Performance gains on GSM8K are modest at the aggregate level, indicating that the current least-to-most design is not a universal improvement over chain-of-thought prompting.
- The approach relies heavily on human prompt engineering and does not provide an automated method for discovering optimal decompositions or prompt structures.

The conclusion suggests that prompting may ultimately be a limited way of teaching reasoning skills to LLMs because it is essentially a one-way communication channel. Future work may aim to transform prompting into richer bidirectional interactions that treat the model as an active participant, with least-to-most prompting viewed as one step toward such interactive teaching.

7.4 Key takeaways

Three important takeaways from the paper are:

1. Explicit problem decomposition combined with sequential subproblem solving can dramatically improve easy-to-hard generalization of LLMs, particularly in symbolic and compositional settings.
2. Least-to-most prompting enables general-purpose LLMs to achieve near-perfect performance on challenging benchmarks such as SCAN length split, without any training or task-specific architectures.
3. The main bottleneck lies in designing decomposition prompts that generalize across tasks and domains, suggesting that future progress will depend on more automated or interactive methods for learning how to decompose complex problems.

Dataset / Task	Domain	Construction / Split
Last-letter-concatenation	Symbolic string task	Synthetic test lists of length 4 to 12

SCAN (length split)

Command to action mapping

Standard SCAN length split